

Complete User Manual

1. Purpose

This manual explains how to reproduce the chromosome-level Haplotype Diversity workflow without assuming prior Linux expertise.

The current analysis begins with a VCF file and performs genotype processing, phasing, LD analysis, haplotype-block analysis, diversity statistics, selection-related scans and population differentiation analysis.

The workflow can be adapted to another chromosome, population, plant species, animal, or other diploid organism provided that the input data and biological assumptions are appropriate.

2. Required knowledge

Basic familiarity with these concepts is sufficient:

- chromosome
- SNP/variant
- genotype
- reference and alternate allele
- VCF
- missing genotype
- haplotype
- population
- linkage disequilibrium
- nucleotide diversity
- Tajima's D
- FST

3. Hardware recommendations

The exact requirements depend strongly on:

- number of variants
- number of samples
- number of chromosomes
- LD-window size
- number of pairwise comparisons
- heatmap size

For a small chromosome/subset, a normal modern workstation can be sufficient.

For large datasets, pairwise LD calculations can become computationally expensive.

Recommended starting configuration:

- 8+ CPU cores
- 16–32 GB RAM
- SSD storage
- Java installed for Beagle
- Python 3.10/3.11 environment

4. Software

Main Python components used by the workflow:

- Python
- pandas
- NumPy
- scikit-allel
- scikit-learn
- SciPy
- Matplotlib
- Seaborn

External software:

- Java
- Beagle

The original notebook installs/uses scikit-learn, Matplotlib, Seaborn and NetworkX and imports scikit-allel, NumPy and pandas.

The original analysis also specifies a Beagle JAR and Java executable path.

5. Installation

5.1 Windows

Open Anaconda Prompt.

Create an environment:

```
conda create -n environment name python=3.11 -y ### (here make your own environment)
```

```
conda activate environment name ### (here write your own environment name)
```

Install packages:

```
python -m pip install numpy pandas scipy scikit-learn matplotlib seaborn networkx scikit-allel jupyterlab pyyaml
```

Start Jupyter:

```
jupyter lab
```

5.2 Linux / WSL

Open Ubuntu/WSL terminal:

```
sudo apt update
```

```
sudo apt install -y git openjdk-17-jre
```

Create the Python environment:

```
conda create -n environment name python=3.11 -y ### (here make your own environment)
```

```
conda activate environment name ### (here write your own environment name)
```

Install packages:

```
python -m pip install numpy pandas scipy scikit-learn matplotlib seaborn networkx scikit-allel jupyterlab pyyaml
```

6. Input VCF

The expected input is a VCF containing:

- chromosome/contig
- genomic position
- reference allele
- alternate allele
- genotype data
- sample identifiers

Example:

#CHROM	POS	ID	REF	ALT	QUAL	FILTER	INFO	FORMAT	SAMPLE1	SAMPLE2 ...
chr1	100	.	A	G	.	PASS	.	GT	0/0	0/1

The original workflow reads:

```
fields=[  
    "variants/CHROM",  
    "variants/POS",  
    "variants/ID",  
    "variants/REF",  
    "variants/ALT",  
    "calldata/GT"  
]
```

7. Step 1 — VCF allele inspection

The workflow first determines whether the VCF contains:

- only biallelic sites
- only multiallelic sites
- a mixture

The function counts total variant sites and separates them into biallelic and multiallelic sites.

Example command:

```
python src/check_vcf.py --vcf data/Your vcf file name.vcf ### (here write your own vcf file name)
```

Expected summary:

Total variant sites; Biallelic sites; Multiallelic sites; VCF type.

This step is important because downstream genotype processing assumes a biallelic representation.

8. Step 2 — Split multiallelic sites

A site such as:

REF=A

ALT=C,G

is converted into separate biallelic markers:

A/C

A/G

The current implementation creates a separate marker for each ALT allele.

For a selected ALT allele:

- reference remains 0
- selected ALT becomes 1
- other ALT alleles are treated as missing

This is a **custom preprocessing decision**, described explicitly in the manuscript.

9. Step 3 — Variant QC

The workflow uses three variant-level filters.

9.1 Minor allele frequency

Current parameter:

MIN_MAF = 0.05

Variants with MAF below 5% are removed.

9.2 Variant missingness

Current parameter:

`MAX_MISSING_VARIANT = 0.10`

Variants with more than 10% missing genotypes are removed.

9.3 Hardy-Weinberg equilibrium

Current parameter:

`HWE_P = 1e-6`

Variants with HWE $P < 1e-6$ are removed.

10. Step 4 — Sample QC

Current parameter:

`MAX_MISSING_SAMPLE = 0.20`

Samples with more than 20% missing genotypes are excluded.

The repository should produce a QC report containing:

samples before QC; samples after QC.

variants before QC; variants after QC.

This is important for reproducibility.

11. Step 5 — KNN genotype imputation

The workflow uses:

```
KNNImputer(  
    n_neighbors=5,  
    weights="distance"  
)
```

Missing genotype values are temporarily represented as NaN.

KNN imputation generates continuous estimates.

The workflow then:

1. rounds to the nearest genotype class
2. clips values to 0–2

3. converts genotype counts back to:

- 0 → 0/0
- 1 → 0/1
- 2 → 1/1

Current setting:

KNN_K = 5

12. Step 6 — Write imputed VCF

The workflow reconstructs a VCF containing GT calls.

Output example:

imputed_knn_Your vcf file name.vcf ### (here your own vcf file name will come)

imputed_knn_Your vcf file name.vcf.gz ### (here your own vcf file name will come)

The pipeline also ensures that a GT FORMAT header is present.

13. Step 7 — Phasing with Beagle

Beagle is called using Java.

Conceptually:

```
java -jar beagle.jar \  
    gt=imputed_knn_Your vcf file name.vcf.gz \ ### (here your own vcf file name will  
    come)  
    out=imputed_knn_phased_Your vcf file name \ ### (here your own vcf file name  
    will come)  
    nthreads=4
```

The exact Beagle version must be recorded.

Do not publish a path such as:

C:\Users\mdipa\Downloads\beagle....jar

Instead configure:

beagle:

jar: "/path/to/beagle.jar"

```
java: "java"
```

```
threads: 4
```

15. Step 8 — Convert phased genotypes to haplotypes

The phased genotype array has dimensions:

variants × samples × ploidy

The workflow converts it to:

variants × haplotypes

For diploid samples:

number of haplotypes = 2 × number of samples

This haplotype matrix is then used for LD and EHH-related calculations.

15. Step 9 — True haplotype LD

The workflow calculates:

- D
- D'
- r^2

For two binary haplotypes:

pA = frequency of allele A

pB = frequency of allele B

pAB = frequency of A and B together

$D = pAB - pA \cdot pB$

Then:

$r^2 = D^2 / [pA(1-pA)pB(1-pB)]$

The code handles cases where the denominator is zero.

This is an explicit haplotype-based implementation and should be described as such.

16. Step 10 — LD decay

The workflow compares variant pairs up to:

`MAX_DISTANCE_BP = 50,000`

and groups distances into:

`BIN_SIZE_BP = 2,000`

For each distance bin, mean r^2 is calculated.

Outputs:

`LD_decay_haplo_Your vcf file name.csv` ### (here your own vcf file name will come)

`LD_decay_haplo_Your vcf file name.png` ### (here your own vcf file name will come)

The CSV contains:

distance_bp; r^2 ; bin; distance_mid.

17. Step 11 — Haplotype blocks

Current thresholds:

`STRONG_DPRIME = 0.8`

`MIN_R2 = 0.2`

`PAIR_COVERAGE = 0.95`

The algorithm extends a block while at least 95% of evaluated pairs satisfy the chosen strong-LD thresholds.

19. Step 12 — Haplotype counts and frequencies

For each block, the workflow creates haplotype codes and counts.

Outputs:

`haplotypes_per_block_Your vcf file name.csv` ### (here your own vcf file name will come)

Columns include: Block; Haplotype; Haplotype_seq; Count; Frequency.

The analysis then selects the maximum-frequency haplotype per block.

19. Step 13 — Recombination landscape

Current settings:

`window = 50 kb`

step = 25 kb

Mean pairwise r^2 is calculated within each window.

Output:

recombination_Your vcf file name.png

20. Step 14 — LD heatmap

A symmetric r^2 matrix is calculated.

Current maximum:

MAX_SNPS_HEATMAP = 300

The value can be increased, but memory/time requirements increase rapidly.

Output:

LD_Haplotype_matrix_Your vcf file name.csv ### (here your own vcf file name will come)

LD_haplotype_heatmap_Your vcf file name.png ### (here your own vcf file name will come)

22. Step 15 — SNP density

SNP density is calculated using: 100 kb windows

Output:

SNP_density_Your vcf file name _100kb.png ### (here your own vcf file name will come)

The result represents the number of SNPs per genomic window.

23. Step 16 — LD scatter plot

All collected pairwise LD values are plotted against physical distance.

Output:

LD_scatter_pairs_Your vcf file name.png ### (here your own vcf file name will come)

For large datasets this can become very large, so sampling or plotting a subset may be appropriate for visualization while retaining the complete CSV.

24. Step 17 — Haplotype-block distributions

The workflow produces:

- block-length histogram
- SNP-count-per-block histogram
- block map

Outputs:

block_length_hist_ [Your vcf file name](#).png **### (here your own vcf file name will come)**

block_snp_count_hist_ [Your vcf file name](#).png

Haplotype_blocks_map_ [Your vcf file name](#).png

25. Step 18 — Haplotype diversity

Unique haplotypes per block are calculated.

Output:

haplotypes_per_block_ [Your vcf file name](#).png

The major haplotype frequency is also plotted:

major_haplotype_frequency_ [Your vcf file name](#).png

26. Step 19 — Shannon entropy of haplotypes

For each block:

$$H = -\sum p \ln(p)$$

where p is haplotype frequency.

Output:

haplotype_entropy_per_block_ [Your vcf file name](#).png

7. Step 20 — FAST EXTENDED HAPLOTYPE HOMOZYGOSITY (EHH)

Extended haplotype homozygosity is calculated around a core SNP.

Current cutoff:

$$\text{EHH_CUTOFF} = 0.05$$

Maximum scan:

$$\text{MAX_WINDOW_SNPS} = 500$$

The algorithm scans left and right and stops when EHH falls below the cutoff.

Output:

EHH_Your vcf file name.png

28. Step 21 — iHS

The workflow calculates:

1. derived-allele EHH
2. ancestral-allele EHH
3. integrated EHH
4. log ratio
5. frequency-bin standardization

The current code uses 20 frequency bins:

```
bins = np.linspace(0, 1, 21)
```

and requires at least 10 SNPs in a bin.

It plots standardized iHS and marks ± 2 .

Output:

iHS_Your vcf file name.png

29. Step 22 — Nucleotide diversity

Genome-wide nucleotide diversity:

π

is calculated using scikit-allel.

Sliding windows:

```
window = 100 kb
```

```
step = 50 kb
```

Output:

nucleotide_diversity_pi_Your vcf file name.png

30. Step 23 — Tajima's D

Genome-wide Tajima's D is calculated.

Sliding-window settings:

`window = 100 kb`

`step = 50 kb`

Output:

`tajimas_d_Your vcf file name.png`

31. Step 24 — FST

The samples divide into two groups:

`pop1 = list(range(0, n_samples // 2))`

`pop2 = list(range(n_samples // 2, n_samples))`